

Proyecto final Autonomous Software Factory

Introducción

El presente informe detalla el análisis y los resultados de aplicar un ciclo de ingeniería de software soportado por inteligencia artificial para resolver un problema real: la falta de automatización y trazabilidad en la gestión de trámites públicos. Este escenario genera cuellos de botella y descontrol institucional, los cuales suelen abordarse con soluciones prematuras e improvisadas.

Para evitarlo, se adoptó un enfoque de **gobernanza ejecutable**, orquestando agentes autónomos que redefinen la concepción, la planificación y la validación del software. En primer lugar, la intervención del **Discovery Agent** aplicó la regla de "cero invención" para descartar supuestos iniciales y definir un Producto Mínimo Viable (MVP) realista. Su análisis permitió pasar de simplemente intentar cambiar el medio de envío de documentos, a resolver la crisis subyacente de seguimiento y priorización de trámites.

Posteriormente, el **Agile Delivery Team** transformó la planificación al separar el trabajo en roles automáticos independientes (Product Owner, Scrum Master, Architect y Developer). La orquestación de estos roles implementó un *gate* de *Definition of Ready* (DoR) para asegurar que cada historia de usuario ingresara al flujo de trabajo cumpliendo estrictamente con los criterios INVEST.

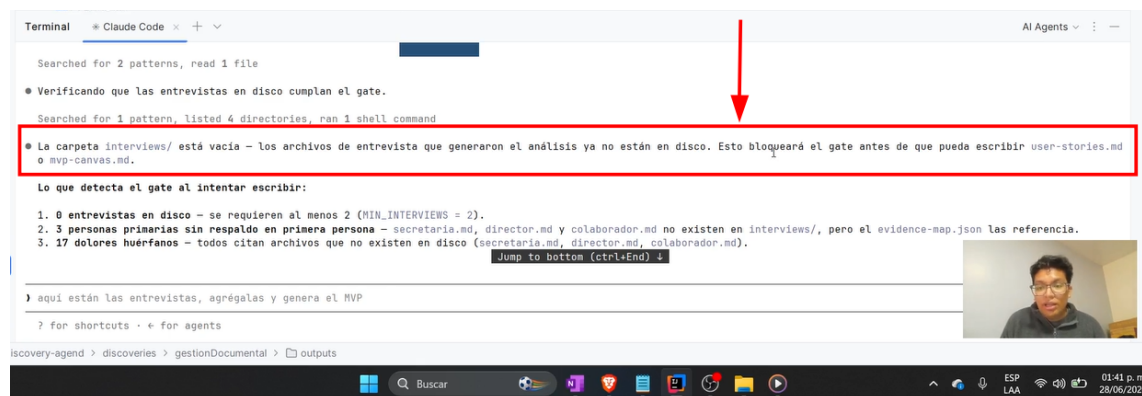
Finalmente, el **Desarrollo con SDD y el Quality Agent** aportaron el rigor técnico en la fase de construcción. Mientras que Spec-Kit guio la implementación basada en especificaciones concretas, el Quality Agent eliminó la subjetividad humana mediante un *gate* determinista de *Definition of Done* (DoD), validando automáticamente las pruebas, los criterios de aceptación y la seguridad del código. En conjunto, la intervención de estos agentes optimiza la creación de software y garantiza que el sistema resultante sea trazable, seguro y verdaderamente útil para mitigar los problemas del usuario en su jornada diaria.

Por consiguiente, la orquestación de estas herramientas es fundamental para el éxito de los sistemas institucionales. No solo optimizan la creación de código, sino que aseguran que el software resultante sea trazable, seguro y verdaderamente útil para el usuario final.

Análisis del Discovery Agent

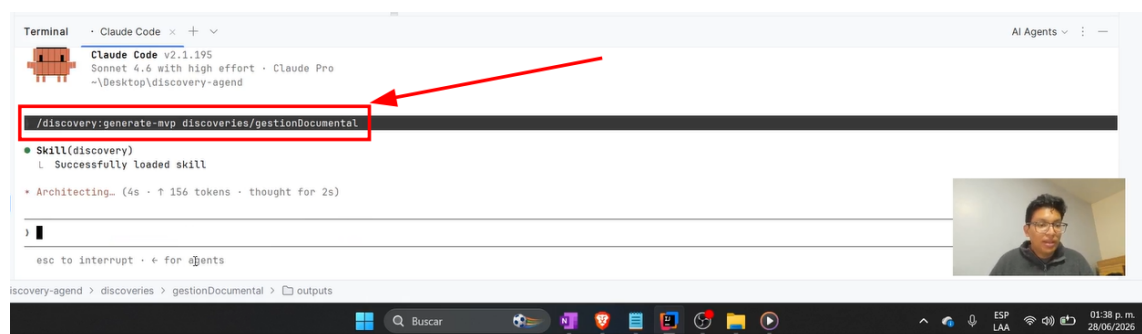
El análisis de esta etapa detalla la inmersión en el contexto real del usuario mediante el procesamiento de evidencias crudas. El Discovery Agent permitió identificar la verdadera causa raíz de los cuellos de botella en la gestión de trámites, enfocando el desarrollo hacia la trazabilidad y descartando supuestos técnicos prematuros bajo la regla estricta de "cero invención".

Evidencia del bloqueo



```
Terminal  Claude Code  + - -  
  
Searched for 2 patterns, read 1 file  
  
• Verificando que las entrevistas en disco cumplan el gate.  
Searched for 1 pattern, listed 4 directories, ran 1 shell command  
  
• La carpeta interviews/ está vacía - los archivos de entrevista que generaron el análisis ya no están en disco. Esto bloqueará el gate antes de que pueda escribir user-stories.md  
  o mvp-canvas.md.  
  
Lo que detecta el gate al intentar escribir:  
1. 0 entrevistas en disco - se requieren al menos 2 (MIN_INTERVIEWS = 2).  
2. 3 personas primarias sin respaldo en primera persona - secretaria.md, director.md y colaborador.md no existen en interviews/, pero el evidence-map.json las referencia.  
3. 17 dolores huérfanos - todos citan archivos que no existen en disco (secretaria.md, director.md, colaborador.md).  
  Jump to bottom (Ctrl+End) >  
  
> aquí están las entrevistas, agrégalas y genera el MVP  
? for shortcuts · ← for agents  
  
iscovery-agent > discoveries > gestionDocumental > outputs
```

comando

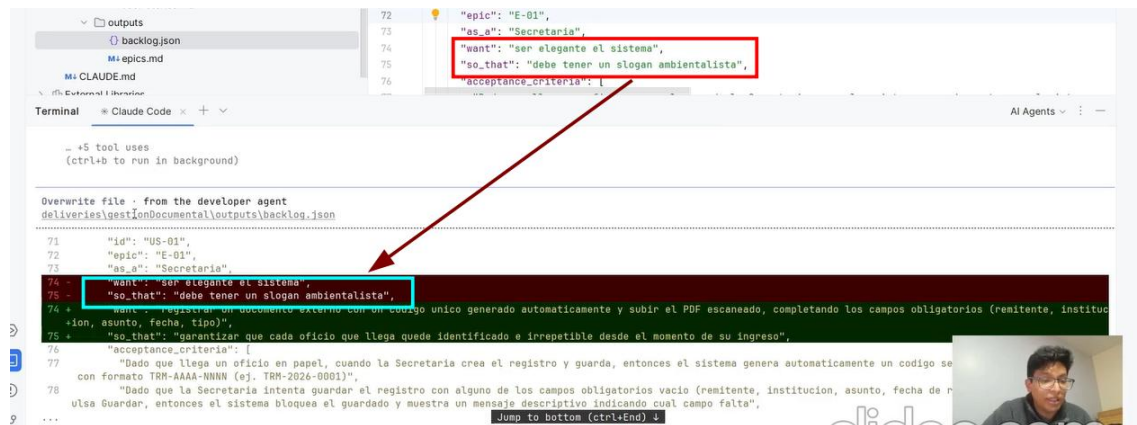


```
Terminal  Claude Code  + - -  
  
Claude Code v2.1.195  
Sonnet 4.6 with high effort · Claude Pro  
~\Desktop\discovery-agent  
  
/discovery:generate-mvp discoveries/gestionDocumental  
  
• Skill(discovery)  
  Successfully loaded skill  
  
• Architecting... (4s · ↑ 156 tokens · thought for 2s)  
  
> █  
  
esc to interrupt · ← for @ents  
  
iscovery-agent > discoveries > gestionDocumental > outputs
```

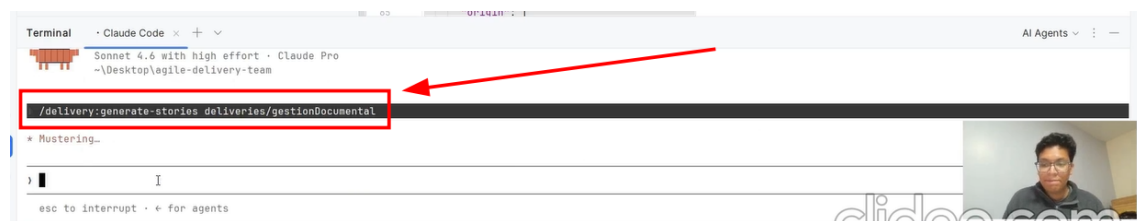
Análisis del Agile Delivery Team

En esta sección se evalúa la automatización de la planificación al dividir el flujo de trabajo en roles especializados (Product Owner, Architect, Scrum Master y Developer). Este equipo autónomo refinó las prioridades y la estructura técnica, garantizando mediante un validador automatizado que cada requerimiento ingresara al desarrollo cumpliendo rigurosamente con los criterios INVEST.

Bloqueo



Comando



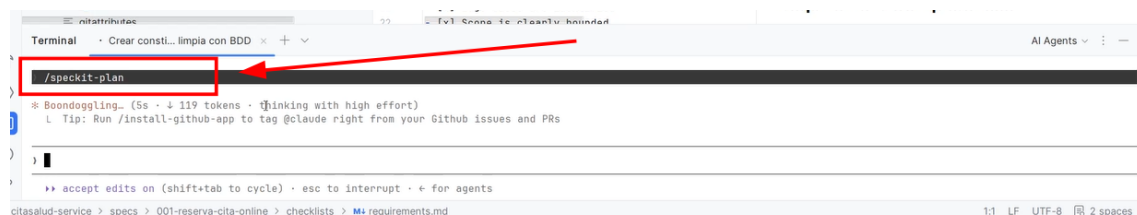
Análisis del desarrollo con SDDKit

Este apartado describe la transición del diseño a la implementación utilizando el Desarrollo Basado en Especificaciones (SDD). Mediante Spec-Kit, se establecieron lineamientos unificados que conectaron directamente los criterios de aceptación del negocio con la arquitectura del código, guiando la construcción de una funcionalidad modular y orientada a pruebas.

Comando 1



Comando 2



Comando 3

```
Terminal  * Crear consti... limpia con BDD  + -
Próximo paso
/speckit-tasks
* Cooked for 5m 22s
) /speckit-tasks Optional task generation constraints
  >> accept edits on (shift+tab to cycle)
citasalud-service > specs > 001-reserva-cita-online > plan.md 1:1 LF UTF-8 4 spaces
```

Comando4

```
Terminal  * Crear consti... limpia con BDD  + -
/speckit-analyze
* Running 1 shell command..
* Swooping... (6s · ↓ 105 tokens)
  ↳ Tip: Use /permissions to pre-approve and pre-deny bash, edit, and MCP tools
) 
```

Comando 5

```
Terminal  * Crear consti... limpia con BDD  + -
* Churned for 1m 46s
/speckit-implement
* Frolicking...
  ↳ Tip: Double-tap esc to rewind the code and/or conversation to a previous point in time
) 
```

Análisis del Quality Agent

El **Quality Agent** funcionó como el guardián automatizado de la entrega, validando que el código no solo compile, sino que cumpla con los estándares de calidad definidos. El uso del comando `quality:verify` y la posterior generación del reporte permitieron una evaluación determinista del *Definition of Done* (DoD)

QUALITY & GOVERNANCE AGENT · UNIDAD 3

Reporte de calidad — gestionDocumental v1.0.0

commit no .git en la ruta auditada · 2026-07-06 · umbral cobertura 80%

APROBADO CON OBSERVACIONES

PRUEBAS PASA

38/38

cobertura línea 100% · rama 94.8%

SEGURIDAD PASA

0 críticas / 0 high

1 medio (autorización simulada) · 0 secretos

CRITERIOS CON OBSERVACIONES

13/14

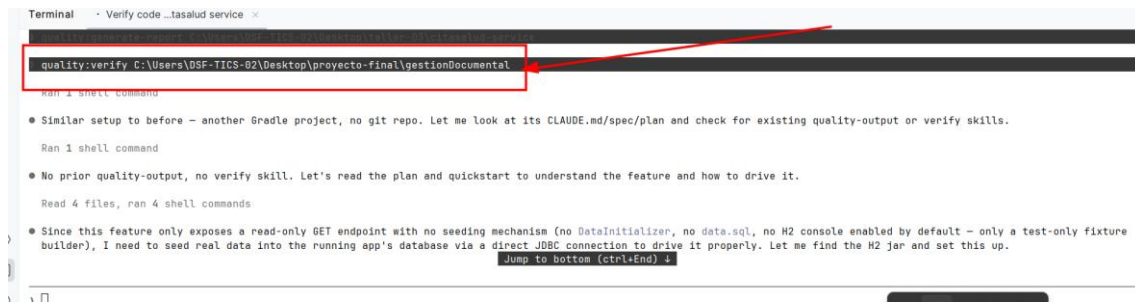
critérios que cumplen (1 con salvedad incluido)

VERIFICACIÓN EN CALIENTE (RUNTIME) · 2026-07-06

El endpoint GET /tramites/{id}/historiales es de solo lectura y no tiene mecanismo de siembra de datos en producción. Se sembraron eventos reales insertando filas vía JDBC directo sobre el archivo H2 que respalda la app (bootRun con spring.datasource.url=jdbc:h2:file:./build/gdocdb) y se ejercitó el controlador REST real de punta a punta. **AC-1, AC-2 y los 4 pasos de AC-3 se comportan exactamente como especifica el spec.** Único hallazgo: los errores 400 (header faltante / UUID inválido) usan un formato de cuerpo distinto al resto de la API y no documentado en el contrato OpenAPI.

CRITERIOS DE ACEPTACIÓN

Comando 1



```
Terminal · Verify code ...tasalud service ×
quality:verify C:\Users\DSF-TICS-02\Desktop\proyecto-final\gestionDocumental

Ran 4 shell commands

• Similar setup to before - another Gradle project, no git repo. Let me look at its CLAUDE.md/spec/plan and check for existing quality-output or verify skills.

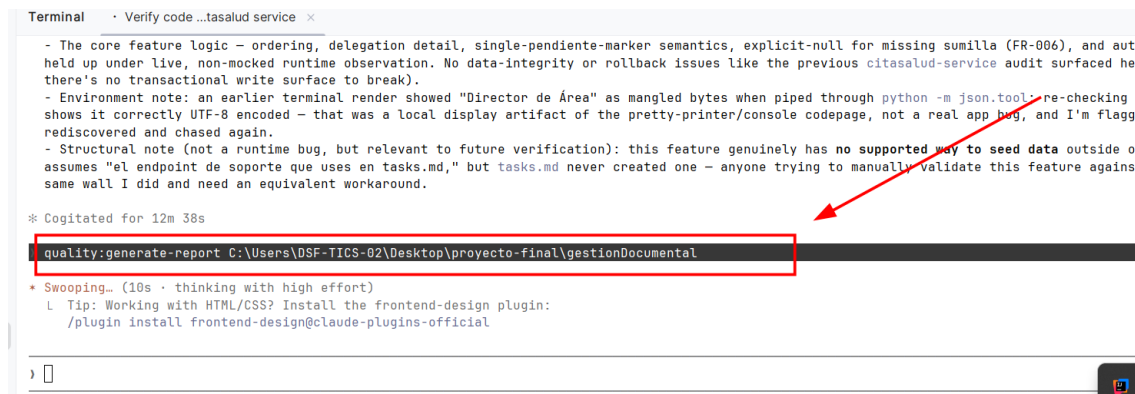
Ran 1 shell command

• No prior quality-output, no verify skill. Let's read the plan and quickstart to understand the feature and how to drive it.

Read 4 files, ran 4 shell commands

• Since this feature only exposes a read-only GET endpoint with no seeding mechanism (no DataInitializer, no data.sql, no H2 console enabled by default - only a test-only fixture builder), I need to seed real data into the running app's database via a direct JDBC connection to drive it properly. Let me find the H2 jar and set this up.
Jump to bottom (ctrl+End) ↓
```

Comando 2



```
Terminal · Verify code ...tasalud service ×
- The core feature logic - ordering, delegation detail, single-pendiente-marker semantics, explicit-null for missing sumilla (FR-006), and auth held up under live, non-mocked runtime observation. No data-integrity or rollback issues like the previous citasalud-service audit surfaced here there's no transactional write surface to break).
- Environment note: an earlier terminal render showed "Director de Área" as mangled bytes when piped through python -m json.tool - re-checking t shows it correctly UTF-8 encoded - that was a local display artifact of the pretty-printer/console codepage, not a real app bug, and I'm flaggi rediscovered and chased again.
- Structural note (not a runtime bug, but relevant to future verification): this feature genuinely has no supported way to seed data outside of assumes "el endpoint de soporte que uses en tasks.md," but tasks.md never created one - anyone trying to manually validate this feature against same wall I did and need an equivalent workaround.

* Cogitated for 12m 38s

quality:generate-report C:\Users\DSF-TICS-02\Desktop\proyecto-final\gestionDocumental

* Swooping... (10s - thinking with high effort)
  Tip: Working with HTML/CSS? Install the frontend-design plugin:
      /plugin install frontend-design@claude-plugins-official
```

Conclusiones

La aplicación del Discovery Agent demostró que resolver un problema público no requiere de software complejo, sino de una arquitectura que responda fielmente a las necesidades reales del usuario mediante la regla de "cero invención".

La automatización de roles en el Agile Delivery Team optimiza la calidad desde la concepción, garantizando que solo los requerimientos técnicamente viables y con valor de negocio superen el gate de Definition of Ready.

El uso del Quality Agent y los gates deterministas es indispensable para garantizar que el software resultante sea robusto, eliminando la subjetividad humana y asegurando estándares de seguridad que son críticos en el sector público.

Recomendaciones

Institucionalización de Gates: Se recomienda integrar los gates de calidad (DoR y DoD) en los repositorios institucionales de la Universidad Técnica de Cotopaxi para estandarizar la entrega de todos los proyectos de software del departamento de TI.

Capacitación en Gobernanza Ejecutable: Es vital promover el uso de herramientas de gobernanza ejecutable para alinear a los equipos de desarrollo con los objetivos reales de la institución, reduciendo la brecha entre los requerimientos administrativos y las soluciones técnicas.

Escalamiento del Calidad Automática: Se sugiere automatizar el escaneo de seguridad (vía MCP) no solo al final del ciclo de vida del software, sino como un proceso de integración continua desde las primeras fases de desarrollo.

Anexos:

<https://github.com/a-gavilanez/discovery-agend>

<https://github.com/a-gavilanez/agile-delivery-team>

<https://github.com/a-gavilanez/proyecto-final>

<https://github.com/a-gavilanez/quality-agend>